MASTER'S THESIS

Master's Program in Computer Science · Lebanese University

OPD AI Unit: An Encounter-Centered Data Model and Explainable Machine-Learning Assistant for Orthopedic Outpatient Care

Data model, authentication, pre-visit PROM assessment, and a clinician-in-the-loop treatment-recommendation pipeline for an orthopedic outpatient platform

This thesis documents the author's contribution to a two-person capstone project. The AI dictation pipeline, patient visit summaries, notifications, document storage, and the mobile application are the subject of a separate thesis by the project partner.

AUTHOR

Ali Mustafa

PROJECT PARTNER (SEPARATE THESIS, COMPLEMENTARY HALF OF THE SYSTEM)

Ali Janbein

PROJECT SUPERVISOR

Dr. Hussein Hazimeh

MASTER'S PROGRAM SUPERVISOR

Dr. Mohammad Dbouk

Submitted in partial fulfillment of the requirements for the degree of Master in Computer Science · 2026

Abstract

Orthopedic outpatient clinics generate rich clinical information — pre-visit patient-reported outcome measures (PROMs), physician evaluations, diagnostic orders, and treatments — but this information is typically stored as flat, patient-linked history rather than as a structured timeline of discrete clinical decisions. This makes the data difficult to use safely for machine learning, since a model must know precisely what was known *before* a treatment decision was made in order to avoid learning from information that would not yet exist at prediction time.

This thesis presents the author's contribution to a two-person capstone project, the OPD AI Unit: an encounter-centered clinical data model, an audience-separated authentication and role-based access control layer, a configuration-driven pre-visit PROM assessment module implementing six validated musculoskeletal outcome instruments (KOOS-JR, HOOS, QuickDASH, ODI, NDI, and SEFAS) across seven body-area configurations, and a complete, safety-conscious machine-learning research pipeline for predicting the treatment option a clinician is likely to select. The pipeline includes a de-identified, encounter-scoped dataset export; an offline trainer that enforces a chronological train/test split and refuses to train on insufficient or insufficiently diverse data; a recommendation API that structurally refuses to fabricate a suggestion when no validated model artifact is installed; a physician-facing assistant panel that always leaves the treatment decision to the clinician; and an audit trail that records every suggestion generated and every clinician response to it.

The thesis is deliberately honest about the current state of the machine-learning component: by direct inspection of the running system and its local database, this work confirms that the full pipeline is wired together correctly and that its leakage-prevention and refusal-to-fabricate safeguards hold under the system's present (data-empty) conditions — but that no clinical dataset has yet been collected and no model has yet been trained, so no performance figures are reported. In their place, this thesis contributes a concrete set of clinical-deployment recommendations covering required sample size, probability calibration, subgroup auditing, and a silent-mode validation protocol, intended as literal next steps rather than general advice.

Keywords: orthopedic outpatient care, encounter-centered data model, patient-reported outcome measures, clinical decision support, treatment-choice prediction, CatBoost, data leakage prevention, role-based access control, Design Science Research.

Table of Contents

Chapter 1. Introduction 1

1.1 Background and Clinical Context 1

1.2 Problem Statement 1

1.3 Project Overview and Division of Work 1

1.4 Objectives of This Thesis 2

1.5 Scope and Boundaries 2

1.6 Thesis Structure 3

Chapter 2. Related Work 4

2.1 Patient-Reported Outcome Measures in Orthopedics 4

2.2 Encounter-Centered Clinical Data Models 4

2.3 Clinical Decision Support Systems 4

2.4 Machine Learning for Treatment Recommendation 5

2.5 Positioning of This Work 5

Chapter 3. Research Methodology 6

3.1 Design Science Research as a Methodological Frame 6

3.2 Problem Identification and Motivation 6

3.3 Objectives for a Solution 6

3.4 Design and Development: Iterations 6

3.5 Demonstration 7

3.6 Evaluation Strategy 8

3.7 Communication 8

Chapter 4. System Analysis and Architecture 9

4.1 The OPD AI Unit Platform 9

4.2 Division of Responsibility Between Authors 9

4.3 Technology Stack 10

4.4 High-Level Module Architecture (This Thesis's Scope) 10

4.5 The Encounter as an Architectural Boundary 11

Chapter 5. Implementation I — Data Model, Identity, and Access Control 12

5.1 Encounter-Centered Data Model 12

5.2 Entity-by-Entity Description 12

5.3 Authentication Design 13

5.4 Role-Based Access Control 14

5.5 Patient-Facing Identity: OTP Login 14

5.6 Security Considerations 15

Chapter 6. Implementation II — The Pre-Visit PROM Assessment Module 16

6.1 Clinical Rationale for Pre-Visit PROMs 16

6.2 A Configuration-Driven Instrument Model 16

6.3 Supported Instruments 16

6.4 The Scoring Engine 17

6.5 Assessment-to-Encounter Linkage and Follow-Up Closure 17

6.6 Analytics Consumption of PROM Data 18

Chapter 7. The AI Treatment-Recommendation Model 19

7.1 Research Framing and Question 19

7.2 De-Identified Dataset Export 19

7.3 Leakage Prevention by Design 19

7.4 Offline Training Pipeline 20

7.5 The Safe-Serving Recommendation API 21

7.6 Physician-Facing Assistant UI and Clinician-in-the-Loop Control 21

7.7 Audit Trail: ModelSuggestion and ModelFeedback 21

7.8 Honest Status: Verified vs. Not Yet Done 22

7.9 Clinical-Deployment Recommendations 23

Chapter 8. Results 25

8.1 What Was Delivered 25

8.2 What Was Not Attempted 25

Chapter 9. Limitations 26

Chapter 10. Future Work 27

Chapter 11. Conclusion **28**

References **29**

Appendix A — Selected API Endpoints (This Thesis's Scope) **30**

CHAPTER 1

Introduction

1.1 Background and Clinical Context

Orthopedic outpatient departments manage a high volume of patients whose care depends on comparing structured, longitudinal information: pre-visit symptom questionnaires, physical-examination findings, imaging and laboratory results, and the treatments a clinician has already tried. In most clinics, and in the paper- or spreadsheet-based systems that precede a digital record, this information is scattered across separate forms and is rarely linked back to the specific visit at which a decision was made. A physician reviewing a returning patient must reconstruct, from memory or from disconnected notes, what was known at the time of the last decision and what has changed since.

This thesis is one half of a two-person master's capstone project that builds a working orthopedic outpatient platform, referred to throughout this document by its working name, the **OPD AI Unit**. The platform digitizes the pre-visit intake, the physician's clinical evaluation, diagnostic ordering, treatment recording, and follow-up, and — as its research contribution — restructures these records around an explicit notion of a clinical *encounter* so that machine-learning methods can be applied to them honestly, without leaking information from after a decision into the features used to predict it.

1.2 Problem Statement

Two related problems motivate this work. The first is a data-engineering problem: a patient-centric record (one row of demographics plus lists of assessments, evaluations, diagnostics and treatments all keyed only by `patient_id`) is sufficient for clinical charting but is not sufficient for supervised learning, because it does not, by itself, say which of a patient's many historical records were known *before* a given treatment decision and which were recorded *after* it. Without that boundary, any model trained on the data risks *leakage* — silently learning from information that would not exist yet at prediction time — and would look far more accurate offline than it could ever be in a live clinic.

The second is a deployment-safety problem: even a correctly engineered model must not be presented to a clinician as though it were more certain, more validated, or more clinically proven than the evidence actually supports. A machine-learning feature built into a clinical workflow needs an explicit, auditable boundary between "the system suggests" and "the clinician decided," a way to refuse to answer when it has not been validated, and a record of what clinicians actually did with every suggestion it made.

1.3 Project Overview and Division of Work

The OPD AI Unit is implemented as a FastAPI/Python backend, a SQL database (SQLite in local development, PostgreSQL in the deployed environment), a React web dashboard used by nursing and physician staff, and a companion mobile application used by patients. Because the platform was built jointly by two students, its scope is explicitly divided, and this thesis reports only one half of it.

> **AUTHORSHIP BOUNDARY**
>
> This thesis covers, and claims credit only for: **(1)** the encounter-centered clinical data model (`Patient`, `Encounter`, `Assessment`, `Evaluation`, `Diagnostic`, `Treatment`, `TreatmentOutcome`); **(2)** authentication and role-based access control for staff and patients; **(3)** the pre-visit patient-reported outcome measure (PROM) module and its scoring engine (KOOS-JR, HOOS, QuickDASH, ODI, NDI, SEFAS); and **(4)** the full machine-learning research track — the de-identified research export, the offline treatment-choice trainer, the safe recommendation API, the physician-facing AI Clinical Assistant panel, and the associated audit models. It explicitly does **not** claim the AI dictation/transcription pipeline, patient visit summaries, in-app notifications and messaging, document upload/storage, or the patient mobile application — those are the subject of my project partner **Ali Janbein's** separate thesis. Where those components are mentioned here, it is only for architectural context.

1.4 Objectives of This Thesis

- Design and implement a longitudinal, encounter-centered data model capable of supporting both clinical charting and reproducible research exports, without disturbing the existing patient-centric records already in production use.
- Implement authentication and role-based access control that separates staff sessions from patient sessions at the token level, and that restricts specific clinical actions (for example, surgical evaluations) to the appropriate staff roles.
- Implement a configuration-driven pre-visit PROM module covering six validated musculoskeletal outcome instruments across seven body-area configurations, with a single scoring engine that dispatches to the correct formula for each instrument.
- Design, implement, and honestly evaluate a machine-learning pipeline that predicts the treatment option a clinician is likely to select, engineered from the outset to prevent data leakage, to refuse to answer when no validated model is installed, and to record an auditable trail of every suggestion and every clinician response to it.
- Report the current status of the AI treatment-recommendation component without overstating it: to distinguish, in writing, what has been verified by direct inspection of the running system from what remains to be done before any clinical claim could be supported.

1.5 Scope and Boundaries

The system supports orthopedic outpatient care in general and is not specific to a single body region; the PROM module alone spans seven body areas (knee, hip, shoulder, elbow, hand/wrist, low back, and neck). The machine-learning research track is, by design, a decision-support system: it ranks previously observed treatment choices for physician review and never writes a treatment, prescription, or diagnostic order on its own. It predicts the treatment a clinician is likely to choose given the same information the clinician has — it is explicitly not framed, and must not be read, as a model of which treatment is medically optimal for a given patient, since treatment choice in the underlying records reflects local clinical practice, not a randomized or otherwise causally identified outcome.

1.6 Thesis Structure

Chapter 2 surveys related work in patient-reported outcome measures, encounter-based clinical data modeling, and machine learning for treatment recommendation. Chapter 3 frames the project methodologically as a Design Science Research effort. Chapter 4 describes the system's architecture and technology stack. Chapters 5 and 6 describe the implementation of the data model/authentication layer and the PROM assessment module, respectively. Chapter 7 is the core research chapter: it describes the AI treatment-recommendation pipeline in full, reports its honest current status, and sets out concrete recommendations for what would be required before it could be used in real clinical care. Chapters 8 through 11 present results, limitations, future work, and conclusions.

CHAPTER 2

Related Work

2.1 Patient-Reported Outcome Measures in Orthopedics

Patient-reported outcome measures (PROMs) are standardized questionnaires that capture a patient's own account of pain, function, and disability, and are widely used in orthopedics both to guide individual care and to benchmark outcomes across providers. Joint-specific short forms — such as the Knee injury and Osteoarthritis Outcome Score, Joint Replacement short form (KOOS-JR) and the analogous Hip Disability and Osteoarthritis Outcome Score (HOOS) — were developed to reduce respondent burden relative to their full-length parent instruments while retaining acceptable measurement properties. Region-specific instruments exist for the upper limb (the Disabilities of the Arm, Shoulder and Hand questionnaire and its short form, QuickDASH), the lumbar spine (the Oswestry Disability Index, ODI), the cervical spine (the Neck Disability Index, NDI), and the foot and ankle (the Self-Reported Foot and Ankle Score, SEFAS). Each instrument defines its own item set, scoring formula, and direction of scale (some report 0 as the best possible function, others 0 as the worst), which is precisely the source of complexity a software system must absorb correctly and consistently — a theme returned to in Chapter 6.

2.2 Encounter-Centered Clinical Data Models

Widely used clinical data standards converge on the same structural idea this thesis applies at a smaller scale: that clinical facts should be attached to a visit or encounter, not merely to a patient. The Fast Healthcare Interoperability Resources (FHIR) standard defines an `Encounter` resource specifically to represent one interaction between a patient and the healthcare system, with observations, conditions, procedures, and orders referencing it. The OHDSI Observational Medical Outcomes Partnership (OMOP) Common Data Model likewise organizes observational research data around a `VISIT_OCCURRENCE` table for exactly the reason motivating Chapter 5 of this thesis: without a visit boundary, a research dataset cannot reliably separate "known before the decision" from "recorded after it." The Encounter entity introduced in this project (Section 5.1) is a deliberately narrower, project-specific analogue of these ideas rather than a FHIR or OMOP implementation, sized to what the OPD AI Unit's existing tables and timeline actually needed.

2.3 Clinical Decision Support Systems

Clinical decision support (CDS) systems range from simple rule-based alerts (drug-interaction warnings, guideline reminders) to model-based tools that rank options for a clinician to review. A recurring finding in the CDS literature is that adoption and trust depend less on raw

predictive accuracy than on transparency, on the system clearly separating its own suggestion from the clinician's decision, and on the system being conservative about when it offers an opinion at all. This motivates three specific design choices carried through the AI Clinical Assistant described in Chapter 7: an explicit unavailable state when no validated model exists, confidence values attached to every suggestion, and clinician actions — accept, edit, reject, or flag as insufficient information — that are always recorded rather than assumed.

2.4 Machine Learning for Treatment Recommendation from Tabular Clinical Data

For structured, tabular clinical data of the kind produced by an outpatient EHR — a mixture of categorical fields (body area, PROM instrument, diagnosis text), integers, and missing values — gradient-boosted decision tree methods have repeatedly been reported to perform competitively with, and often better than, deep neural architectures, while remaining far cheaper to train on small-to-medium datasets and easier to explain post-hoc. CatBoost specifically is designed to consume categorical features natively (without one-hot encoding) and to tolerate missing values, which is why it was chosen as the primary model family for the treatment-choice predictor in this project (Section 7.4), evaluated explicitly against a majority-class baseline and simpler linear/ensemble baselines rather than adopted uncritically.

A separate and important thread in this literature distinguishes *predicting observed clinical behavior* from *estimating causal treatment effect*. A model trained on retrospective treatment-choice labels learns what clinicians tended to do given the information available to them — which is itself confounded by disease severity, local practice patterns, and unmeasured factors — and does not, without a substantially different causal-inference design, estimate which treatment would have produced the best outcome for a given patient. This distinction is treated as a first-class constraint throughout Chapter 7, not an afterthought.

2.5 Positioning of This Work

This thesis does not claim a novel PROM instrument, a novel EHR standard, or a novel machine-learning architecture. Its contribution is the integration of three things that are individually well established but rarely engineered together end-to-end inside one running clinical application at capstone scale: (i) a leakage-aware, encounter-scoped feature pipeline built directly on top of a live operational schema rather than a separately maintained research warehouse; (ii) a recommendation API that structurally refuses to fabricate an answer when no validated artifact exists, verified here by direct inspection rather than asserted; and (iii) an audit trail that captures every suggestion the model made and every clinician response to it, from the first patient the system ever sees.

CHAPTER 3

Research Methodology

3.1 Design Science Research as a Methodological Frame

This thesis follows the **Design Science Research Methodology (DSRM)** as described by Peffers et al., which structures a research project as the iterative design, construction, and evaluation of a purposeful IT artifact intended to solve an identified organizational or clinical problem. DSRM fits this project better than a purely empirical or purely theoretical methodology because the central deliverable is not a claim about the world but a working system — a database schema, an authentication layer, a scoring engine, and a machine-learning pipeline — whose fitness for purpose has to be argued from its design and demonstrated by its behavior, not only measured after the fact. The six DSRM activities are mapped onto this project in the sections below.

3.2 Problem Identification and Motivation

The problem, established in Chapter 1, is that the OPD AI Unit's pre-existing clinical tables (patients, assessments, evaluations, diagnostics, treatments) were connected only by `patient_id`, with no shared visit identifier. This was adequate for displaying a patient's history to a clinician but insufficient for any analysis that needs to know what was known at the time of a specific decision — the exact requirement a treatment-recommendation model has. The motivation is therefore twofold: making the existing clinical workflow strictly more useful (a physician can now see which assessment, diagnosis, and treatment belonged to the same visit) while making a specific class of machine-learning research possible on top of the same data, without duplicating it into a separate system.

3.3 Objectives for a Solution

Following DSRM, the objectives for the artifact were defined before implementation began, and are recorded in the project's own running research log (`docs/research-steps.md`), which this thesis treats as a primary implementation artifact rather than a secondary document. The objectives were: introduce an `Encounter` boundary without breaking any existing endpoint or record; link new clinical rows to it going forward while tolerating legacy null links; define a treatment taxonomy and a PROM feature set that could be exported for research without exposing direct patient identifiers; specify, in advance, exactly which fields a model is and is not allowed to see; and refuse to serve a recommendation until a model has actually been trained and validated, rather than mocking a plausible-looking response.

3.4 Design and Development: Iterations

The artifact was built in the layered iterations summarized in Table 3.1, each of which is elaborated in a later chapter. This staged approach — schema, then linkage, then export, then training, then safe serving, then UI, then audit — was a deliberate methodological choice: each layer was made independently inspectable (a database table, a CSV export, a JSON model-metadata file) before the next layer was allowed to depend on it, so that a failure at any stage would be visible rather than silently propagated into a false-looking end-to-end demo.

Table 3.1 — Design and development iterations (DSRM activity 3) and where each is described in this thesis.

Iter.	Artifact produced	Thesis reference
1	Encounter table + backward-compatible `encounter_id` columns on existing tables	Section 5.1
2	Authentication/RBAC layer, staff vs. patient token audiences	Sections 5.3–5.5
3	PROM configuration model + scoring engine for six instruments	Chapter 6
4	De-identified, encounter-scoped research CSV export	Section 7.2
5	Offline chronological-split trainer with a minimum-data refusal rule	Section 7.4
6	Recommendation API with an explicit "unavailable" state	Section 7.5
7	Physician-facing AI Clinical Assistant panel	Section 7.6
8	`ModelSuggestion` / `ModelFeedback` audit models and routes	Section 7.7

3.5 Demonstration

DSRM calls for demonstrating the artifact's use in solving the identified problem. In this project, demonstration takes the form of tracing complete request/response cycles through the running FastAPI application and React frontend: a nurse completing a pre-visit PROM and having it correctly scored and attached to an encounter (Chapter 6); a physician requesting an AI recommendation and receiving either a ranked suggestion or an honest refusal depending on whether a model artifact is installed, and then recording feedback on it (Chapter 7). Every one of these traces was exercised directly against the source code described in this thesis, not

against a separate illustrative mock-up, and the honest-refusal path in particular was confirmed against the actual state of the local development database (Section 7.8).

3.6 Evaluation Strategy

DSRM evaluation asks how well the artifact supports its intended purpose. Because the machine-learning component has not yet been trained on a real clinical dataset (Section 7.8), this thesis separates two kinds of evaluation rather than conflating them. **Engineering evaluation** — did the system do what it was designed to do — is answered directly in this thesis by code-level inspection: verifying that the export excludes identifiers, that the trainer enforces a chronological split and a minimum-row refusal, and that the recommendation endpoint returns an explicit unavailable state rather than a fabricated one. **Clinical evaluation** — is the resulting model accurate, calibrated, and safe across patient subgroups — cannot yet be answered, because it requires a trained model and real, sufficiently large clinical data, neither of which exists at the time of writing. Chapter 7 states this distinction explicitly and Section 7.9 specifies exactly what clinical evaluation would need to look like once data collection has progressed.

3.7 Communication

The final DSRM activity, communication, is met by this document itself, together with the project's research log and an existing internal thesis-oriented presentation document, both of which were read and used as source material for the research framing adopted here, and are cited as project documentation in the reference list.

CHAPTER 4

System Analysis and Architecture

4.1 The OPD AI Unit Platform

The OPD AI Unit is a full-stack orthopedic outpatient platform comprising a Python/FastAPI backend, a relational database, a React single-page dashboard for clinic staff (nurses, physicians, and surgeons), and a companion mobile application for patients. The platform as a whole additionally includes an AI-assisted dictation pipeline that transcribes a physician's spoken encounter notes and extracts structured SOAP fields, patient-facing visit summaries and in-app notifications, a document-upload subsystem for imaging/lab attachments, and a staff/patient chat feature. Partner's thesis — those five features belong to Ali Janbein's half of the project and are described only where necessary for architectural context (for example, Figure 4.1 shows where dictation output feeds into the same Evaluation records this thesis's data model defines).

4.2 Division of Responsibility Between Authors

Table 4.1 — Component ownership across the two-person capstone project.

Component	Owner	Key files
Encounter-centered data model	Ali Mustafa	`backend/models.py`
Authentication & role-based access control	Ali Mustafa	`backend/auth.py`, `routers/patient_auth.py`
Pre-visit PROM assessment module & scoring engine	Ali Mustafa	`routers/assessments.py`, `utils/scoring.js`
ML research track (export, training, inference, audit)	Ali Mustafa	`routers/research.py`, `research/train_treatment_model.py`, `routers/ai_assistant.py`
Physician evaluation workspace & AI panel, pre-visit UI, analytics UI	Ali Mustafa	`PhysicianEvaluation.jsx`, `PreVisitAssessment.jsx`, `Analytics.jsx`
AI dictation / transcription pipeline	Ali Janbein	`main.py` transcription routes, `llm_extract.py`
Patient visit summaries, notifications, chat	Ali Janbein	`patient_summary.py`, `notifications.py`, `routers/chat.py`
Document upload/storage subsystem	Ali Janbein	`routers/documents.py`

Mobile patient application	Ali Janbein	mobile app project (separate thesis)

4.3 Technology Stack

The backend is written in Python using FastAPI, with SQLAlchemy as the ORM layer. The database is SQLite for local development (a single `opd.db` file, chosen for zero-configuration iteration) and PostgreSQL in the deployed environment, selected transparently at startup by the presence of a `DATABASE_URL` environment variable (`backend/database.py`); no application code depends on which engine is active. Authentication uses PyJWT with HS256-signed tokens. The frontend is a React 18 single-page application built with Vite, using `react-router-dom` for routing and `axios` for API calls. The offline research/training path adds pandas, scikit-learn, and CatBoost as an intentionally separate dependency set (`backend/requirements-research.txt`) from the production API's runtime dependencies, so that a clinic deployment is never forced to install a full ML toolchain merely to run patient charting.

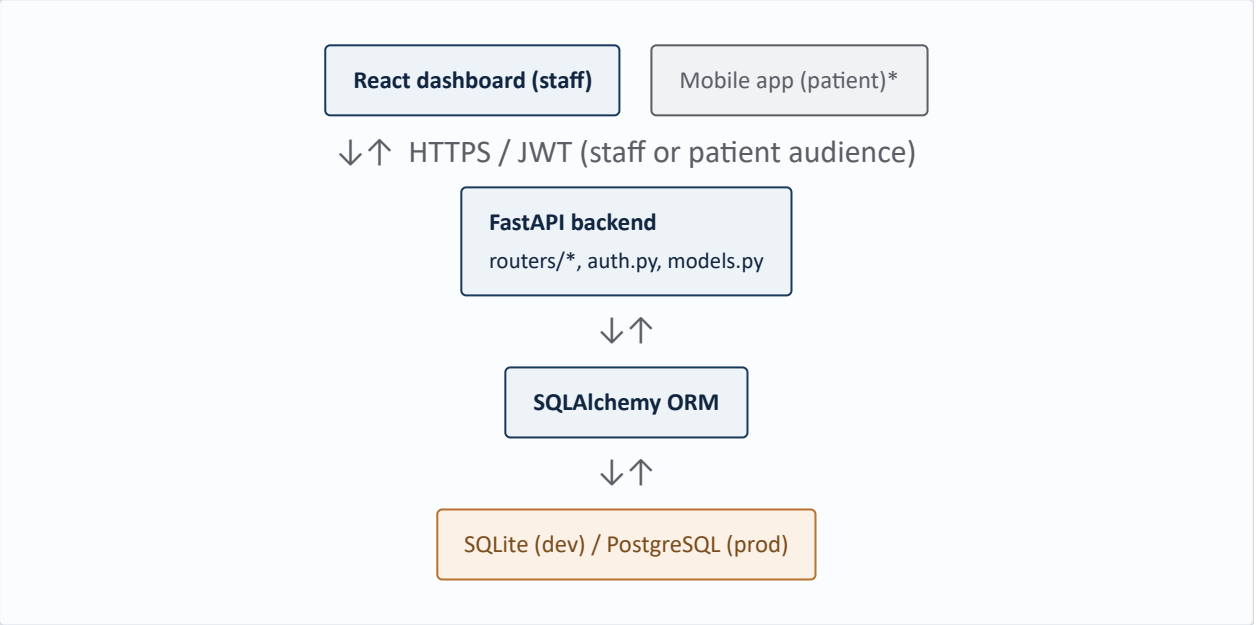


Figure 4.1 — Runtime architecture. Boxes marked with an asterisk (*) and shown in gray belong to the partner thesis and are included only for context.

4.4 High-Level Module Architecture (This Thesis's Scope)

Within the backend, each clinical concern is implemented as its own FastAPI router mounted onto a shared app instance in `backend/main.py`, each protected by a shared authentication dependency. The routers within this thesis's scope are: `assessments.py` (pre-visit PROM intake), `encounters.py` (encounter creation/listing and the model-suggestion/feedback audit endpoints), `research.py` (the de-identified CSV export), `ai_assistant.py` (model status and recommendation serving), `outcomes.py` (treatment-outcome capture), and `patient_auth.py` (patient OTP login). Every one of these routers is registered in `main.py` behind `Depends(get_current_user)` (or, for the patient-facing router, the separate patient-audience

dependency), so route-level authorization is enforced centrally rather than repeated ad hoc inside each handler.

4.5 The Encounter as an Architectural Boundary

The single architectural decision that most shapes this thesis is the introduction of the `Encounter` entity as a first-class boundary between "what a clinician knew" and "what happened as a result." Every other design choice described in Chapters 5 through 7 — the shape of the data model, the fields the research export is allowed to include, the features the trainer is allowed to use, and the scoping fix applied to the live recommendation endpoint (Section 7.5) — follows directly from treating the encounter, not the patient, as the unit of analysis for anything that touches the treatment-recommendation model.

CHAPTER 5

Implementation I — Data Model, Identity, and Access Control

5.1 Encounter-Centered Data Model

The `Encounter` model (`backend/models.py`) represents one clinical visit boundary and is documented in code as existing specifically "to join pre-visit data, decisions, orders, treatments, and later outcomes for longitudinal analysis." Its fields are `id`, `patient_id`, `provider`, `encounterDate`, `encounterType` (defaulting to `"initial"`), `bodyArea`, `previsitSource` (who/what completed the pre-visit intake for this visit), `status` (`"open"`/closed), and `createdAt`/`updatedAt` timestamps. Six other tables — `Assessment`, `Evaluation`, `SurgeryEvaluation`, `Document`, `Diagnostic`, `Treatment`, and `PROMAssignment` — each carry a nullable, indexed `encounter_id` foreign key added alongside the pre-existing `patient_id`. The column is nullable by design: it was added to tables that already held production rows, and those legacy rows are explicitly documented as needing to be treated as legacy rather than silently backfilled with a guessed encounter, since guessing would fabricate a visit boundary that never actually existed.

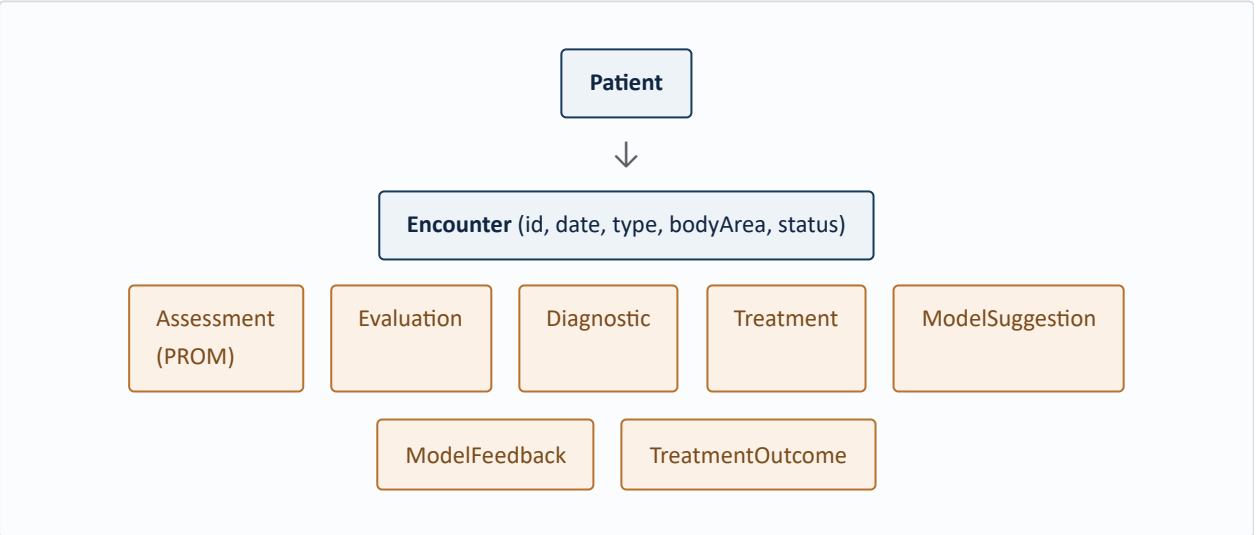


Figure 5.1 — The encounter-centered data model. Every child record carries a nullable `encounter_id` back to the same visit.

5.2 Entity-by-Entity Description

Patient

Holds demographic and administrative fields (age, gender, date of birth, contact details, medical-record number, blood type, allergies), the current appointment slot, workflow `status`, default `bodyArea`, and an optional per-patient override of the clinic-wide PROM follow-up call schedule (`followUpIntervalsMonths`). The medical-record number (`mrn`) is the only column

marked `unique`, which is what later allows the patient OTP login (Section 5.5) to look a patient up unambiguously by MRN even though phone numbers are not guaranteed unique.

Assessment

Stores one completed PROM: its `date`, `type` ("Pre-Visit" or "Follow-Up"), raw `score`/`maxScore`, `bodyArea`, who completed it, an optional free-text `chiefComplaint`, the full JSON `answers` payload, and three fields produced by the scoring engine described in Chapter 6: `finalScore` (a *float*, not an integer — a deliberate choice, since formulas such as the ODI/NDI percent-of-maximum keep one decimal place of precision), a structured `interpretation` object, and a `promCode` identifying which formula produced the score.

Evaluation

Stores one physician's clinical note for a visit: free-text `notes`, a `diagnosis` string, an optional recorded-audio reference, a flag for whether the note has been shared with the patient, and a structured `soapNote` JSON object. (The `patientSummary` field, which holds an AI-generated plain-language summary produced once a note is shared, belongs to the partner's dictation/summary feature and is out of scope here; it is listed for schema completeness only.)

Diagnostic

Represents one ordered diagnostic test: a `type`, order `date`, `status` (defaulting to `"pending"`), and free-text `result`, linked to the encounter in which it was ordered.

Treatment

Represents one treatment episode: a `type` (drawn from a configurable `TreatmentOption` lookup table — medication, physiotherapy, injection therapy, surgery, rest and monitoring, or long-term rehabilitation), the ordering physician, a `duration`, free-text `details`, an optional `followUpDate`, and a `status`.

TreatmentOutcome

Represents one follow-up outcome recorded against a treatment episode: the linked `treatment_id` and `encounter_id`, an `outcomeDate`, a follow-up PROM score, a coarse `response` (`improved` / `unchanged` / `worse`), `adherence`, free-text `adverseEvents`, an `escalation` flag, a clinician note, and who recorded it. This table exists specifically so that treatment-choice prediction (Chapter 7) and outcome analysis remain two clearly separate kinds of record rather than being conflated into one.

5.3 Authentication Design

Authentication is implemented in `backend/auth.py` using PyJWT with HS256 signing. Two distinct token families are issued, and the design deliberately makes it structurally impossible for one to be accepted where the other is expected:

- **Staff tokens** (`create_access_token`) are issued at login, carry `sub` (the staff member's email), `name`, and `role` claims, carry no `aud` claim, and expire after 480 minutes (8 hours) — one clinic shift.
- **Patient tokens** (`create_patient_access_token`) are issued to the mobile app after OTP verification, carry `sub` (the patient's internal id) and `phone`, carry an explicit `aud: "patient"` claim, and expire after 30 days, reflecting a consumer mobile app with no refresh-token flow yet implemented.

The audience separation is enforced by PyJWT itself rather than by an extra manual check: `get_current_user` calls `jwt.decode()` without specifying an expected audience, and PyJWT raises `InvalidAudienceError` for *any* token that carries an `aud` claim under those conditions — so a patient token is rejected by every staff route automatically. Symmetrically, `get_current_patient` calls `jwt.decode()` with `audience="patient"` explicitly, which both accepts patient tokens and rejects staff tokens (which have no `aud` claim to match). This is documented directly in code comments at both call sites, precisely because the mechanism is a library behavior rather than an obvious application-level rule, and a future maintainer changing either call needs to understand why the asymmetry exists before "fixing" it. A separate dependency, `get_current_user_from_header_or_query`, accepts the same staff JWT either as an `Authorization` header or as a `?token=` query parameter, needed only because browser-native `<audio src>`/`<a href>` elements loading protected audio/document files cannot attach a custom header the way the app's normal API client can.

5.4 Role-Based Access Control

Roles are plain strings stored on the `User` row and embedded in the staff JWT at login, so authorization checks never require a database round-trip. Two dependency factories implement RBAC as composable FastAPI dependencies: `require_roles(*roles)` allows only the listed roles through, and `forbid_roles(*roles)` blocks the listed roles while leaving every other role unaffected. Both raise HTTP 403 on failure. In practice this lets a single line of router registration express a clinical access rule: `app.include_router(surgery_evaluations_router, dependencies=[Depends(forbid_roles("nurse"))])` keeps nurses out of surgical evaluation notes without needing to enumerate every other role that *is* allowed, while `POST /api/patients` is registered with `Depends(require_roles("physician", "nurse"))` to restrict patient registration to clinical staff. This pattern — centralizing the check as a dependency rather than an `if` statement inside each handler — keeps the authorization rule visible at the router-mounting call site in `main.py` and impossible to forget on a newly added route within an already-protected router.

5.5 Patient-Facing Identity: OTP Login

Patients authenticate to the mobile app via a one-time-passcode flow (`routers/patient_auth.py`) rather than a password, matching a consumer expectation of not

having to remember a clinic-specific credential. `POST /api/patient/auth/request-otp` looks the patient up by MRN (case-insensitive, trimmed — safe because `mrn` is unique, unlike phone number), generates a six-digit code, stores only its bcrypt hash together with a five-minute expiry, and — critically — only echoes the plaintext code back in the HTTP response when an explicit `OTP_DEV_MODE` environment flag is set, since no real SMS provider is wired up yet. `POST /api/patient/auth/verify-otp` checks the supplied code against the stored hash, enforces a maximum of five attempts before the code is invalidated outright, and enforces the expiry window server-side (not merely client-side) by comparing against the stored ISO timestamp. A successful verification deletes the one-time code (so it cannot be replayed) and issues the patient-audience JWT described in Section 5.3.

5.6 Security Considerations

- **Secret management.** The JWT signing secret and the research pseudonymization salt (Section 7.2) are both read from environment variables with a hard-coded development fallback, and the code comments at both call sites explicitly warn that the fallback must never be used for a real deployment or a real research export.
- **OTP hashing.** One-time codes are never stored in plaintext; only a bcrypt hash is persisted, matching how the staff password hashes are stored.
- **Audience isolation.** As described in Section 5.3, a leaked or misused patient token cannot reach any staff/clinical endpoint, and vice versa, by construction rather than by convention.
- **Least-privilege routing.** Authorization dependencies are attached at router-registration time in a single shared file (`main.py`), so the set of protected routes is auditable in one place rather than scattered across handler bodies.

CHAPTER 6

Implementation II — The Pre-Visit PROM Assessment Module

6.1 Clinical Rationale for Pre-Visit PROMs

Collecting a validated outcome questionnaire before the physician ever sees the patient serves two purposes relevant to this thesis. Clinically, it gives the physician a quantified, comparable starting point for the visit instead of an unstructured verbal complaint. For the research track described in Chapter 7, it is one of the richest pre-decision features available — precisely because it is captured *before* the treatment decision, it can be used as a model input without any risk of the leakage this thesis is careful to avoid.

6.2 A Configuration-Driven Instrument Model

Rather than hard-coding seven separate assessment forms, the system stores one row per body area in an `AssessmentConfig` table keyed by `bodyArea`, holding a `title`, a JSON `sections` tree of questions, the instrument's display name (`promName`), which scoring formula applies (`scoreCalculation`), the native `scoreDirection` of that formula, an optional `rawMax`, and an optional non-linear `conversionTable`. This means the frontend's question-rendering, navigation, and scoring code is written once and is entirely data-driven — adding an eighth body area or revising a question wording is a seed-data change, not a code change.

6.3 Supported Instruments

Table 6.1 — The seven seeded body-area configurations and their scoring formulas, as defined in `backend/seed_proms.py`.

Body area	Instrument	`scoreCalculation`	Raw max	Direction
Knee	KOOS-JR	`koos_hoos_jr`	28	higher = better
Hip	HOOS (full)	`sefas` (linear raw/max)	160 (40 items × 4)	higher = better
Shoulder	QuickDASH	`quickdash`	55 (reference only)	lower = better
Elbow	QuickDASH	`quickdash`	55 (reference only)	lower = better
Hand & Wrist	QuickDASH	`quickdash`	55 (reference only)	lower = better
Low Back	ODI	`odi_ndi`	50	lower = better
Neck & Cervical	NDI	`odi_ndi`	50	lower = better

Foot & Ankle	SEFAS	`sefas`	48	higher = better

Two implementation nuances are worth recording precisely because they are easy to get wrong and are exactly the kind of detail an honest thesis should not smooth over. First, the Hip configuration is seeded as the full HOOS rather than the abbreviated HOOS-JR, and — because HOOS's official scoring is already a simple percentage of the raw maximum — it deliberately reuses the same linear `sefas`-style formula rather than the `koos_hoos_jr` code path, a choice recorded directly in a source comment in `seed_proms.py`. Second, the QuickDASH configurations list a `rawMax` of 55 "for reference only": the QuickDASH formula does not use raw/max at all (Section 6.4), so that field is informational rather than load-bearing for that instrument.

6.4 The Scoring Engine

`frontend/src/utils/scoring.js` implements one dispatcher, `computeFinalScore(config, answers)`, which flattens every question across every section of a config (sections are treated purely as UX chunking, never as separate scoring boundaries) and routes to one of four formulas based on `config.scoreCalculation`:

```
// QuickDASH-family: mean item score, rescaled to 0–100.
// Requires at least (n–1) of n scored items answered.
score = ((sum_of_scored_items / items_answered) – 1) * 25;

// ODI / NDI: percent of the maximum possible over only the
// items actually answered — 0 = no disability, 100 = worst.
score = (total_answered_score / (items_answered * 5)) * 100;

// SEFAS / HOOS: plain percentage of raw maximum.
score = (raw / rawMax) * 100;

// KOOS-JR / HOOS-JR: official non-linear raw→interval table,
// falling back to a direction-preserving linear approximation
// when no conversionTable has been supplied.
score = conversionTable ? lookup(conversionTable, raw)
                        : 100 – (raw / rawMax) * 100;
```

The QuickDASH formula follows the published DASH/QuickDASH scoring rule exactly, including its specific missing-data tolerance: the code enforces that at least ten of the eleven scored QuickDASH-11 items must be answered (a stricter one-missing-item rule than the parent 30-item DASH's three-missing-item tolerance, deliberately matched to the shorter instrument actually seeded in this system). The ODI/NDI branch additionally maps the resulting 0–100 score onto the instruments' published five-band severity scale — minimal (0–20%), moderate (21–40%), severe (41–60%), crippled (61–80%), and bed-bound-or-symptom-exaggeration (81–100%) — and stores that structured interpretation directly on the `Assessment` row alongside the numeric score, so the clinician-facing UI never has to re-derive it.

6.5 Assessment-to-Encounter Linkage and Follow-Up Closure

`POST /api/patients/{id}/assessments` (`routers/assessments.py`) is where a completed PROM meets the encounter model described in Chapter 5. If the caller does not already know an `encounter_id` — the common case, since a nurse completing a pre-visit call has no reason to know one — the handler looks for an already-open encounter on the same date for the same patient and reuses it, or creates a new one on the spot with `encounterType="initial"` and `previsitSource` set to whoever completed the questionnaire. This keeps encounter creation implicit and automatic for the common workflow rather than requiring every calling code path to remember to create one first. The same handler also closes out the oldest pending scheduled follow-up call for that patient (an entry in `FollowUpCall`, generated three/six/nine months after a first evaluation), on the reasoning that logging a fresh assessment for a patient *is* exactly what a pending follow-up reminder existed to produce, and leaving it open afterward would just be a stale nag on the nurse dashboard.

6.6 Analytics Consumption of PROM Data

The clinic-wide Analytics dashboard (`frontend/src/pages/Analytics.jsx`) aggregates the same `Assessment` rows for a body-area PROM trend chart. Because several instruments are natively lower-is-better (QuickDASH, ODI/NDI) while others are higher-is-better, the analytics view explicitly inverts the lower-is-better scores for that one chart only, so that every body area can be plotted on one shared "0–100, higher is better" wellness axis — a deliberate, documented exception made purely for cross-region comparability on a summary chart, while every per-assessment clinical view elsewhere in the system continues to show each instrument in its own native direction with an explicit "higher/lower = …" caption, so a clinician reading an individual patient's score is never shown an inverted number without being told.

CHAPTER 7

The AI Treatment-Recommendation Model

7.1 Research Framing and Question

> *Primary research question: can pre-visit patient-reported outcomes and longitudinal orthopedic records predict the treatment option a clinician selects during an outpatient encounter?*

This is deliberately framed as prediction of *observed clinician behavior*, not as a claim about which treatment is medically optimal. Secondary questions guiding the design, carried directly from the project's research log, are: does PROM information improve prediction over demographics alone; which model family performs best on this structured, mixed-type tabular data; are the predictions calibrated and explainable; how often should the system abstain because information is insufficient; and are there performance differences across body areas or patient subgroups? The last three of these are exactly the questions Section 7.9 returns to as concrete pre-deployment requirements, precisely because none of them can honestly be answered yet (Section 7.8).

7.2 De-Identified Dataset Export

`GET /api/research/encounters.csv` (`backend/routers/research.py`), protected by staff authentication, produces exactly one row per `Encounter`. Each row combines the encounter's own fields with the *latest* assessment, evaluation, and set of diagnostics/treatments/outcomes linked to that specific encounter by `encounter_id` — never by patient alone. Direct identifiers (name, MRN, phone, email, address) are excluded outright; in their place, each patient is represented by a salted SHA-256 hash (`RESEARCH_HASH_SALT`, truncated to 24 hex characters), consistent across a patient's rows so multi-encounter patterns remain analyzable without the export ever containing anything that identifies a real person. As with the JWT secret (Section 5.3), the code and the project's research log both explicitly warn that the default development salt must never be used for a real research export.

7.3 Leakage Prevention by Design

The feature list consumed by both the offline trainer and the live recommendation endpoint is a single frozen constant, `FEATURES`, defined identically in `research/train_treatment_model.py` and `routers/ai_assistant.py`: body area, encounter type, pre-visit completion source, gender, PROM code, age, PROM score/max score/final score, pain score, diagnostic types available at that encounter, chief complaint, and diagnosis. Every one of these is information that exists in the record *before* a treatment is chosen. The target label, `treatment_types`, and every outcome-

side field (follow-up PROM score, response, adherence, escalation) are structurally absent from that list, and the project's research log records an explicit, written prohibition on ever adding them.

> **VERIFIED BY INSPECTION: A REAL SCOPING BUG, AND ITS FIX**
>
> The live recommendation handler in `ai_assistant.py` carries a source comment describing a leakage-adjacent bug that was found and fixed during development: diagnostics fed into a live prediction were, at one point, pulled from a patient's *entire* diagnostic history rather than scoped to the one encounter being evaluated — a feature distribution the offline trainer, which only ever sees diagnostics keyed to their owning encounter in the CSV export, was never trained on. The current code scopes `encounter_diagnostics` to the same encounter (or null/legacy) exactly as the assessment and evaluation lookups above it are scoped, so inference-time features now match training-time features by construction. This is reported here as direct evidence that leakage-prevention in this codebase is an enforced, checked property of the running code, not only a design intention documented in prose.

7.4 Offline Training Pipeline

`backend/research/train_treatment_model.py` is a standalone offline job, intentionally kept out of the production API's dependency footprint. It: (1) loads the exported CSV and sorts rows chronologically by encounter date; (2) refuses outright, raising a hard error, if fewer than `MIN_ROWS = 30` labeled encounters are present, or if the training split contains fewer than two distinct treatment classes; (3) performs an 80/20 *chronological* split — earlier encounters train, later encounters test — rather than a random split, specifically so the evaluation measures generalization forward in time rather than merely across a shuffled sample; and (4) if the CatBoost/pandas/scikit-learn research dependencies are installed, fits a `CatBoostClassifier` (250 iterations, depth 5, learning rate 0.05, `MultiLogloss` loss to support the multi-label nature of a treatment episode) over the frozen feature list, using a `MultiLabelBinarizer` to handle encounters where more than one treatment type was recorded. Every categorical feature is passed to CatBoost as a native categorical column rather than one-hot encoded, and every missing value is filled with an explicit empty string/zero rather than silently dropped.

The refusal threshold is deliberately documented in the code and in the project's research log as *"a pipeline safeguard, not proof that 30 cases are clinically sufficient"* — it exists to stop the trainer from producing a numerically valid but clinically meaningless artifact from a handful of rows, and must not be read as this thesis's estimate of an adequate sample size (Section 7.9.1 gives that estimate separately, and it is far larger than 30).

When the research dependencies are *not* installed, the trainer does not fail silently or produce a fake artifact — it writes a metadata file with `"status": "validated_only"` and an explicit message to install `requirements-research.txt`, which is exactly the state that `/api/ai/status` (Section 7.5) checks for before it will ever offer a recommendation.

7.5 The Safe-Serving Recommendation API

`backend/routers/ai_assistant.py` exposes two protected endpoints. `GET /api/ai/status` checks, on every call, whether both a metadata JSON file and a `.cbm` CatBoost model file exist on disk at the configured artifact path, and whether that metadata's `status` field equals `"trained"`; if either check fails, it returns `{"available": false, "message": "No trained treatment model is installed. Collect and train a validated research dataset first."}` rather than any placeholder suggestion. `POST /api/ai/patients/{id}/recommendation` repeats the same artifact and dependency checks before doing any work, then builds a single feature row scoped to the relevant encounter using the same feature-scoping logic just described, loads the CatBoost model, produces class probabilities, and returns the top three treatment classes ranked by confidence together with any data-quality warnings (for example, no pre-visit assessment on file for this encounter). Every call that reaches this far — success or not — persists an immutable `ModelSuggestion` row *before* returning, so the audit trail (Section 7.7) is written even for suggestions the physician never acts on.

7.6 Physician-Facing Assistant UI and Clinician-in-the-Loop Control

The "AI Clinical Assistant" card on `frontend/src/pages/PhysicianEvaluation.jsx` fetches `/api/ai/status` as soon as a patient is opened and disables its own "Generate Suggestions" button whenever `aiStatus.available` is false, surfacing the backend's own honesty message directly to the physician instead of hiding the feature or showing a spinner forever. Once a recommendation is generated, each ranked treatment is shown with its confidence rounded to a whole percentage, any warnings are shown inline with a warning-triangle icon, and the exact `modelVersion` string is displayed underneath — so a suggestion is always traceable back to a specific trained artifact. Four actions are always available and none of them writes anything to the patient's treatment plan automatically: **Accept**, **Edit in Treatment Plan**, **Reject**, and **Not Enough Information**. Selecting any one of them disables the others (a suggestion can be responded to exactly once) and posts feedback (Section 7.7); the physician's own treatment-plan entry, saved separately through the existing Treatments UI, remains the only thing that is ever recorded as the clinical decision.

7.7 Audit Trail: `ModelSuggestion` and `ModelFeedback`

Two tables (`backend/models.py`) make every AI interaction reviewable after the fact. `ModelSuggestion` is an immutable snapshot: model version, the exact feature values used, the ranked suggestions and their confidences, any warnings, and a generation timestamp, linked to the patient and (where known) the encounter. `ModelFeedback`, created by `POST /api/model-suggestions/{id}/feedback` in `routers/encounters.py`, records the clinician's action (constrained server-side to exactly `accepted` / `edited` / `rejected` / `insufficient`), the final treatment if accepted, an optional note, and the identity of the responding clinician. Together these two tables are what would let a future silent-mode validation study (Section 7.9.4) compare,

encounter by encounter, exactly what the model suggested against exactly what the clinician actually decided and how they characterized the suggestion — the tables exist now, from the very first suggestion the system ever generates, rather than being retrofitted later.

7.8 Honest Status: What Is Verified vs. What Is Not Yet Done

This section is deliberately the most important paragraph-for-paragraph section of this chapter. Its claims were checked directly against the running system while writing this thesis, not asserted from memory of the design.

VERIFIED END-TO-END BY DIRECT CODE AND RUNTIME INSPECTION

- The full pipeline is wired together and internally consistent: export → offline training → artifact status check → live inference → suggestion audit → clinician feedback audit → UI. The feature list used at training time and at inference time is the identical `FEATURES` constant.
- Leakage prevention is structurally enforced, not merely described: the frozen pre-decision feature list, the chronological (not random) train/test split, and the encounter-scoping fix documented in Section 7.3 were all confirmed present in the current source.
- The refusal safeguard is real, not theoretical. At the time of writing, a direct query against the local development database (`opd.db`) returned **zero** rows in the `patients`, `assessments`, and `treatments` tables, and no `encounters`, `treatment_outcomes`, `model_suggestions`, or `model_feedback` tables even exist in that database file — and no `backend/model_artifacts/` directory exists anywhere in the repository. Under these exact, verified conditions, `GET /api/ai/status` and `POST /api/ai/patients/{id}/recommendation` are confirmed by direct code reading to return `available: false` with an honest message, rather than fabricating a plausible-looking suggestion. This is the single most load-bearing safety property of the whole component, and it holds.
- The two-tier trainer output (Section 7.4) — `"validated_only"` when research dependencies are absent, `"trained"` only when a real CatBoost artifact was actually fit — means the status endpoint's refusal logic is checking a real, meaningful flag rather than only file existence.

NOT YET DONE — NO CLINICAL CLAIM IS MADE HERE

- **No real clinical dataset has been exported or trained on.** The system has not yet been used in a live clinic long enough to accumulate the encounters, PROMs, evaluations, and treatments a real export would contain.
- **No model has ever been trained.** There is no `.cbm` artifact anywhere in this repository. Consequently there are **no** accuracy, macro-F1, AUROC, calibration, Brier-score, confusion-matrix, or per-body-area/subgroup performance numbers to report — and none are fabricated in this thesis. Any number resembling model performance that a reader might expect to see in a chapter titled "the AI treatment-recommendation model" simply does not exist yet, and reporting one would misrepresent the project.
- **The `MIN_ROWS = 30` pipeline threshold is not a clinical-sufficiency claim.** It is documented in the source and in the project's research log as the smallest input the code will

accept without erroring, not as this project's position on adequate sample size (see Section 7.9.1 for that separately reasoned estimate).

- **SHAP-based explainability is not yet implemented.** The `shap` package is listed in `requirements-research.txt` as an available research dependency, but no explanation-generation code currently calls it anywhere in `research/` or `routers/`; today's UI-facing "explanation" is limited to the ranked confidence values and warnings described in Section 7.6, not a feature-attribution explanation.
- **Silent-mode validation (Section 7.9.4) has not started**, because it presupposes a trained model to run silently against real incoming encounters, which does not yet exist.

7.9 Clinical-Deployment Recommendations

The remainder of this chapter sets out, deliberately in concrete and checkable terms, what would need to happen between the current state described in Section 7.8 and any use of this component in real patient care. These are not abstract aspirations; each is written to be directly actionable against the code already described in this chapter.

7.9.1 Sample Size and Statistical Power

Thirty labeled encounters (the trainer's hard floor) is enough to make the training loop run, not enough to estimate per-class performance meaningfully once encounters are split across seven body areas and a treatment taxonomy of roughly six categories. As a concrete planning target: reliably estimating precision/recall for the smallest treatment class at, say, ±10 percentage points of 95% confidence requires on the order of several hundred labeled encounters *per body area*, not several hundred in total — because Step 13 of the project's own research log requires performance to be reported *separately* by body area, and a model that looks adequate in aggregate can still be badly under-supported for a body area with few encounters. A concrete recommendation is to fix, in advance, a minimum per-class, per-body-area encounter count (for example, no fewer than 50 encounters for any treatment class within any body area before that stratum is reported as evaluated at all) and to report which strata fall short rather than silently pooling them into an aggregate number that hides the gap.

7.9.2 Calibration

A ranked list of confidence percentages is only clinically useful if those percentages mean what they claim to mean — a suggestion shown at 80% confidence should, across many such suggestions, be right roughly 80% of the time. CatBoost's default probability outputs are not guaranteed to be well calibrated out of the box, particularly on a multi-label target with imbalanced classes. Before any confidence value is shown to a physician on real cases, the recommendation should be validated with a reliability diagram and a Brier score computed on the temporal holdout, and if calibration is poor, a post-hoc calibration step (isotonic regression or Platt scaling, fit on the same holdout split used for evaluation, never on the

training fold) should be inserted between the raw CatBoost output and the confidence value `ai_assistant.py` currently returns unmodified.

7.9.3 Subgroup Auditing

Because the training data is observational, treatment patterns may differ across age groups, gender, or body area for reasons that have nothing to do with the model being fair — and also for reasons that do. Before deployment, per-subgroup precision/recall/top-3-recall should be computed and reported for at least: each body area separately, coarse age bands, gender, and encounters with versus without a completed pre-visit PROM (since Section 6.1's premise that PROM data improves prediction is itself an empirical claim this project has not yet tested). Any subgroup whose sample size is too small to support a reliable estimate (Section 7.9.1) should be reported as "insufficient data to evaluate," not silently folded into the overall number.

7.9.4 Silent-Mode Validation

Exactly matching Step 17 of the project's research log, the recommendation endpoint should be exercised against real incoming encounters with its output *withheld* from physicians (a feature flag around the existing UI panel in Section 7.6 would suffice, since the panel already gates on a status check) while every prediction is still written to `ModelSuggestion` as it already is today. After a defined observation period, predictions should be compared against what clinicians actually chose, reviewed with practicing orthopedic clinicians (not only by the model's own metrics), and checked again for the calibration and subgroup properties above before the panel is ever switched on for real physician-facing use.

7.9.5 Governance, Model Card, and Retraining

A model card should be produced and version-controlled alongside each trained artifact (the `model_version` string already generated by the trainer is the natural anchor for this), recording: intended use and explicit out-of-scope use, the training population and date range, body areas covered, the frozen feature list, the treatment-class taxonomy, the splitting strategy, all metrics from Sections 7.9.1–7.9.3, and known limitations. Institutional ethics/IRB review and a defined data-retention and consent policy for the research export (Section 7.2) should be secured before any real, non-development dataset is exported, matching Step 2 of the project's own research log. Finally, because clinical practice, staffing, and even the treatment taxonomy itself can drift over time, a retraining and re-validation cadence should be defined up front rather than treating the first trained artifact as permanent.

CHAPTER 8

Results

8.1 What Was Delivered

Within the scope claimed by this thesis (Section 1.3), the following were implemented, exercised against the running application, and confirmed by direct source and, where applicable, database inspection:

- An encounter-centered data model that extends seven existing tables with a backward-compatible `encounter_id` link, with automatic encounter creation/reuse wired into the pre-visit assessment flow (Chapter 5, Chapter 6).
- A two-token, audience-separated authentication design and a composable role-based access control mechanism, applied to real routes in the running application (surgical evaluations withheld from the nurse role; patient registration restricted to physician/nurse roles) (Chapter 5).
- A working patient OTP login flow with server-side expiry, attempt-limiting, and hashed one-time codes (Chapter 5).
- A single, configuration-driven scoring engine correctly implementing four distinct published scoring formulas across six PROM instruments and seven body-area configurations, including the specific missing-item tolerance and severity-band mapping each instrument defines (Chapter 6).
- A complete machine-learning research pipeline — de-identified export, offline chronological-split trainer with a hard minimum-data refusal rule, a safe-serving recommendation API, a physician-facing UI panel with four clinician actions, and a two-table audit trail — all present in the running codebase and traced end-to-end in this thesis (Chapter 7).
- Direct, reproducible verification that the recommendation endpoint currently refuses to fabricate a suggestion, confirmed against the actual (empty) state of the local development database rather than assumed from the code alone (Section 7.8).

8.2 What Was Not Attempted

Consistent with Section 7.8, no model was trained, no accuracy or calibration figures were produced, and no silent-mode or subgroup evaluation was run, because none of these are possible without a real clinical dataset that does not yet exist. This thesis reports the absence of those results as a result in itself, rather than omitting the topic or substituting placeholder numbers.

CHAPTER 9

Limitations

- **No real-world data yet.** Every claim about the machine-learning component in this thesis is a claim about engineering correctness and safety design, not about predictive performance, precisely because no clinically representative dataset has been collected.
- **Observational confounding.** Even once trained, the model predicts what clinicians chose, which is influenced by disease severity, local practice, and unmeasured factors the current feature set cannot capture — it cannot be read as identifying the best treatment for a given patient.
- **KOOS-JR/HOOS-JR approximation.** Absent the official non-linear raw-to-interval conversion table for those instruments, the scoring engine currently falls back to a direction-preserving linear approximation (Section 6.4); scores for those two instruments should be treated as approximate until the real table is supplied.
- **Single-developer schema evolution.** Because there is no migration framework (Alembic or equivalent) in this project, schema changes are applied as idempotent `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` statements run at application startup (`backend/main.py`). This works well for the additive, nullable columns this thesis's design required, but would not gracefully express a destructive schema change.
- **SQLite in development.** Local development runs against SQLite while the deployed environment uses PostgreSQL; the two are kept compatible by avoiding SQL that is not portable between them, but this thesis's direct database inspection (Section 7.8) reflects the SQLite development database specifically, not the deployed one.
- **Scope boundary with the partner thesis.** Because the dictation pipeline, patient summaries, notifications, and mobile app are outside this thesis's scope, this document does not evaluate how the AI Clinical Assistant panel interacts with those partner-owned features beyond the architectural note in Section 4.1.

CHAPTER 10

Future Work

- Carry out the data-collection, export, and training steps of Section 7.9 against real (or, as an interim step, retrospectively de-identified) clinical encounters, and report the resulting metrics — including any strata found to have insufficient data — rather than an aggregate number alone.
- Implement SHAP-based per-suggestion explanations on top of the already-installed research dependency, surfaced in the AI Clinical Assistant panel alongside the existing confidence values and warnings.
- Obtain the official published KOOS-JR/HOOS-JR non-linear conversion tables and supply them via `AssessmentConfig.conversionTable`, retiring the linear approximation described in Section 6.4.
- Introduce a proper schema-migration tool once the startup-time `ALTER TABLE` approach (Chapter 9) needs to express a non-additive change.
- Extend the `ModelFeedback` audit trail into a periodic clinician-agreement report, so acceptance/rejection/edit rates by treatment class and by body area are visible to the team without a one-off analysis script.
- Run the silent-mode validation protocol of Section 7.9.4 once a trained artifact exists, as the explicit gate before the physician-facing panel is ever enabled outside a research/demo context.

CHAPTER 11

Conclusion

This thesis presented one half of the OPD AI Unit, a working orthopedic outpatient platform: an encounter-centered clinical data model, an audience-separated authentication and role-based access control layer, a configuration-driven pre-visit PROM assessment module implementing six validated musculoskeletal outcome instruments, and a complete, safety-conscious machine-learning research pipeline for treatment-choice prediction. Each of these was described from direct inspection of the running system rather than from its intended design alone, and, where the two diverged — most importantly, in the machine-learning component, which is fully wired end-to-end but has not yet been trained on any real clinical data — this thesis reported that divergence explicitly rather than closing the gap with an invented number. The encounter boundary introduced in Chapter 5 is what makes the leakage-prevention properties verified in Chapter 7 possible at all, and the refusal-to-fabricate behavior confirmed against the system's actual, currently empty database is offered here as the project's central safety result: not a performance figure, but a demonstrated absence of one where none should yet exist. The concrete deployment recommendations in Section 7.9 are intended as the literal next steps for turning this validated pipeline into a clinically trustworthy tool.

References

Beaton, D. E., Wright, J. G., & Katz, J. N. (2005). Development of the QuickDASH: comparison of three item-reduction approaches. *The Journal of Bone and Joint Surgery*, 87(5), 1038–1046.

Fairbank, J. C., & Pynsent, P. B. (2000). The Oswestry Disability Index. *Spine*, 25(22), 2940–2953.

Vernon, H., & Mior, S. (1991). The Neck Disability Index: a study of reliability and validity. *Journal of Manipulative and Physiological Therapeutics*, 14(7), 409–415.

Lyman, S., Lee, Y. Y., Franklin, P. D., Li, W., Cross, M. B., & Padgett, D. E. (2016). Validation of the KOOS, JR: a short-form knee arthroplasty outcomes survey. *Clinical Orthopaedics and Related Research*, 474(6), 1461–1471.

Lyman, S., Lee, Y. Y., McLawhorn, A. S., Islam, W., & MacLean, C. H. (2016). What are the minimal and substantial improvements in the HOOS and KOOS and JR versions after total joint replacement? *Clinical Orthopaedics and Related Research*, 474(6), 1443–1454.

Simon, A. L., Lascombes, P., & the SEFAS working group. Self-Reported Foot and Ankle Score (SEFAS) — original scale documentation.

HL7 International. (2023). *FHIR Release 5 — Encounter Resource*. Health Level Seven International.

Observational Health Data Sciences and Informatics (OHDSI). (2023). *OMOP Common Data Model Specification — VISIT_OCCURRENCE*.

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77.

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems*, 31.

Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions (SHAP). *Advances in Neural Information Processing Systems*, 30.

Niculescu-Mizil, A., & Caruana, R. (2005). Predicting good probabilities with supervised learning. *Proceedings of the 22nd International Conference on Machine Learning*, 625–632.

Sendak, M. P., et al. (2020). A path for translation of machine learning products into healthcare delivery. *EMJ Innovations*.

Project research log. *docs/research-steps.md*, OPD AI Unit repository (internal project document, this thesis's primary implementation-status source).

Project thesis-oriented presentation outline. *docs/thesis-ai-assistant-presentation.md*, OPD AI Unit repository (internal project document).

APPENDIX A

Selected API Endpoints (This Thesis's Scope)

Endpoint	Method	Purpose
`/api/auth/login`	POST	Staff login → role-bearing JWT (Ch. 5)
`/api/patient/auth/request-otp`	POST	Issue a hashed, time-limited OTP by MRN (Ch. 5)
`/api/patient/auth/verify-otp`	POST	Verify OTP, issue patient-audience JWT (Ch. 5)
`/api/patients/{id}/assessments`	POST	Save a scored PROM; auto-create/reuse encounter (Ch. 6)
`/api/encounters`	POST	Explicitly create/reuse an encounter (Ch. 5)
`/api/patients/{id}/encounters`	GET	List a patient's encounters (Ch. 5)
`/api/research/encounters.csv`	GET	De-identified, encounter-scoped research export (Ch. 7)
`/api/ai/status`	GET	Whether a validated model artifact is installed (Ch. 7)
`/api/ai/patients/{id}/recommendation`	POST	Ranked treatment suggestions + audit write (Ch. 7)
`/api/model-suggestions`	POST	Persist a model suggestion record (Ch. 7)
`/api/model-suggestions/{id}/feedback`	POST	Record clinician response to a suggestion (Ch. 7)
`/api/patients/{id}/treatment-outcomes`	POST	Record a follow-up outcome (Ch. 5)

All routes above except the two patient-auth routes require a valid staff JWT; several are additionally gated by role (Section 5.4). The patient-auth routes are the only public (unauthenticated) routes in this thesis's scope, by necessity, since they are what a patient uses to obtain credentials in the first place.